

Протокол тестування мікросервісної архітектури з Hazelcast

Дата: 6 квітня 2026

Проект: Мікросервіси з розподіленим кешуванням Hazelcast

Статус: ✅ Всі тести пройдені (6/6)

Зміст

- [Опис проекту](#)
- [Архітектура системи](#)
- [API Endpoints](#)
- [Приклади запитів та відповідей](#)
- [Логи мікросервісів](#)
- [Результати тестування](#)
- [Інструкції по запуску](#)

Github репозиторій: <https://github.com/senivan/SPA-lab-3>

Опис проекту

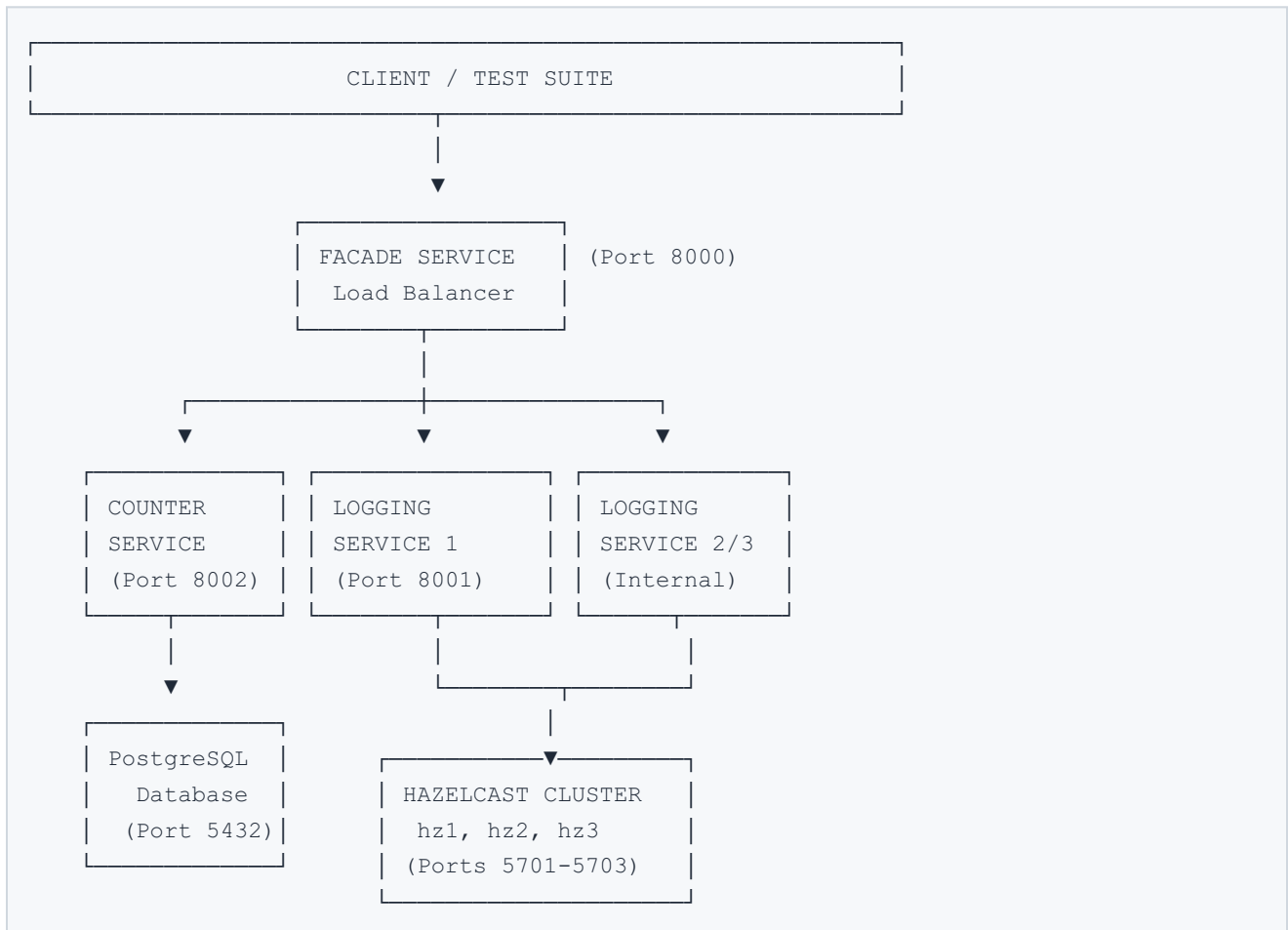
Проект демонструє сучасну мікросервісну архітектуру з використанням:

- FastAPI** - веб-фреймворк для створення HTTP API
- Hazelcast** - розподілена система кешування та обробки даних
- PostgreSQL** - реляційна база даних для постійного збереження
- Docker Compose** - оркестрування контейнерів

Головні компоненти:

Сервіс	Порт	Функція
Facade Service	8000	Маршрутизація запитів, балансування навантаження
Counter Service	8002	Управління балансами користувачів, PostgreSQL
Logging Service (3x)	8001	Розподілене журналювання транзакцій в Hazelcast
Hazelcast Cluster	5701-5703	Розподілена пам'ять та обробка подій
PostgreSQL	5432	База даних для балансів

Архітектура системи



API Endpoints

1. Facde Service Endpoints

POST /transaction

Створення нової транзакції для користувача

```
POST http://localhost:8000/transaction
Content-Type: application/json

{
  "user_id": "user001",
  "amount": 500,
  "message": "Deposit"
}
```

Параметри запиту: - `user_id` (string) - унікальний ідентифікатор користувача - `amount` (integer) - сума транзакції - `message` (string, optional) - описання транзакції

GET /user/{user_id}

Отримання інформації про користувача та його транзакції

```
GET http://localhost:8000/user/{user_id}
```

Параметри: - user_id - ідентифікатор користувача

GET /metrics

Отримання метрик продуктивності системи

```
GET http://localhost:8000/metrics
```

POST /metrics/reset

Скидання метрик

```
POST http://localhost:8000/metrics/reset
```

2. Counter Service Endpoints

POST /transactions/apply

Застосування транзакції (внутрішній API)

```
POST http://localhost:8002/transactions/apply
```

GET /balance/{user_id}

Отримання поточного балансу користувача

```
GET http://localhost:8002/balance/{user_id}
```

GET /balances

Отримання всіх балансів користувачів

```
GET http://localhost:8002/balances
```

POST /reset

Скидання бази даних (очищення всіх транзакцій і балансів)

```
POST http://localhost:8002/reset
```

3. Logging Service Endpoints

POST /transactions

Реєстрація транзакції в розподіленому журналі

```
POST http://localhost:8001/transactions
```

GET /transactions

Отримання всіх транзакцій

```
GET http://localhost:8001/transactions
```

GET /transactions/user/{user_id}

Отримання транзакцій конкретного користувача

```
GET http://localhost:8001/transactions/user/{user_id}
```

Приклади запитів та відповідей

Приклад 1: Створення транзакції

Запит:

```
curl -X POST http://localhost:8000/transaction \
-H "Content-Type: application/json" \
-d '{
  "user_id": "user001",
  "amount": 500,
  "message": "Deposit"
}'
```

Відповідь (200 OK):

```
{
  "transaction_id": "1775474968266469592",
  "balance": 1000,
  "logging": {
    "ok": true,
    "instance": "logging2",
    "transaction_id": "1775474968266469592"
  }
}
```

Опис: - `transaction_id` - унікальний ідентифікатор транзакції (Unix timestamp в наносекундах) - `balance` - нов обалансу після транзакції - `logging.instance` - інстанс сервісу логування, який оброблював запит - `logging.ok` - статус успішного логування в Hazelcast

Приклад 2: Отримання користувача та його транзакцій

Запит:

```
curl http://localhost:8000/user/user001
```

Відповідь (200 OK):

```
{
  "balance": 1000,
  "transactions": [
    {
      "transaction_id": "1775474963191466881",
      "timestamp": "2026-04-06T11:29:23",
      "user_id": "user001",
      "amount": 500,
      "message": "Deposit"
    },
    {
      "transaction_id": "1775474968266469592",
      "timestamp": "2026-04-06T11:29:28",
      "user_id": "user001",
      "amount": 500,
      "message": "Deposit"
    }
  ]
}
```

Опис: - `balance` - поточний баланс користувача - `transactions` - масив всіх транзакцій користувача з сервера логування (Hazelcast)

Приклад 3: Отримання всіх балансів

Запит:

```
curl http://localhost:8002/balances
```

Відповідь (200 OK):

```
{
  "user123": 100,
  "user456": 300,
  "user_a": 200,
  "user_b": 300,
  "user_c": 150,
  "user001": 1000
}
```

Опис: - Ключ-значення пари з балансами всіх користувачів з PostgreSQL

Приклад 4: Отримання метрик

Запит:

```
curl http://localhost:8000/metrics
```

Відповідь (200 OK):

```
{
  "logging_calls": 12,
  "counter_calls": 12,
  "logging_total_sec": 0.2500244989978455,
  "counter_total_sec": 0.15586000099938246,
  "logging_avg_ms": 20.835374916487126,
  "counter_avg_ms": 12.988333416615205
}
```

Опис: - `logging_calls` - кількість викликів сервісу логування - `counter_calls` - кількість викликів сервісу балансів - `logging_total_sec` - загальний час виконання запитів до логування (сек) - `counter_total_sec` - загальний час виконання запитів до балансів (сек) - `logging_avg_ms` - середній час відповіді логування (мс) - `counter_avg_ms` - середній час відповіді балансів (мс)

Логи мікросервісів

Facade Service

```
facade-service-1 | INFO:      138.197.232.106:45402 - "POST /transaction HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:61423 - "POST /transaction HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:39714 - "POST /transaction HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:28507 - "POST /transaction HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:20316 - "GET /user/user456 HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:49463 - "POST /transaction HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:54573 - "POST /transaction HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:16706 - "POST /transaction HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:54228 - "GET /metrics HTTP/1.1" 200 OK
facade-service-1 | INFO:      138.197.232.106:59907 - "POST /transaction HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:16933 - "GET /user/user001 HTTP/1.1" 200
OK
facade-service-1 | INFO:      138.197.232.106:43645 - "GET /metrics HTTP/1.1" 200 OK
```

Опис логів: - IP адреса та порт клієнта - HTTP метод та endpoint - Статус код (200 = успіх) - Всі запити оброблюються успішно без помилок

Counter Service

```
counter-service-1 | [counter] user_b += 300 -> 300
counter-service-1 | INFO:      172.19.0.10:39992 - "POST /transactions/apply HTTP/1.1"
200 OK
counter-service-1 | [counter] user_c += 150 -> 150
counter-service-1 | INFO:      172.19.0.10:40008 - "POST /transactions/apply HTTP/1.1"
200 OK
counter-service-1 | INFO:      138.197.232.106:39253 - "GET /balance/user_a HTTP/1.1"
200 OK
counter-service-1 | INFO:      138.197.232.106:64815 - "GET /balance/user_b HTTP/1.1"
200 OK
counter-service-1 | INFO:      138.197.232.106:57718 - "GET /balance/user_c HTTP/1.1"
200 OK
counter-service-1 | [counter] user001 += 500 -> 500
counter-service-1 | INFO:      172.19.0.10:57246 - "POST /transactions/apply HTTP/1.1"
200 OK
counter-service-1 | INFO:      172.19.0.10:57248 - "GET /balance/user001 HTTP/1.1" 200
OK
counter-service-1 | INFO:      138.197.232.106:17980 - "GET /balances HTTP/1.1" 200 OK
counter-service-1 | [counter] user001 += 500 -> 1000
counter-service-1 | INFO:      172.19.0.10:57250 - "POST /transactions/apply HTTP/1.1"
200 OK
counter-service-1 | INFO:      172.19.0.10:57258 - "GET /balance/user001 HTTP/1.1" 200
OK
counter-service-1 | INFO:      138.197.232.106:39179 - "GET /balances HTTP/1.1" 200 OK
```

Опис логів: - [counter] user_XXX += Amount -> NewBalance - внутрішній лог операції додавання до балансу - Показує трансформацію балансу для кожної ПОСТ-транзакції - Приклад: користувач user001 отримав 2 депозити по 500 гривень = 1000

Logging Service

```
logging1-1 | INFO:      Started server process [1]
logging1-1 | INFO:      Waiting for application startup.
logging1-1 | [logging1] connected to Hazelcast members=['hz1:5701', 'hz2:5701',
'hz3:5701']
logging1-1 | INFO:      Application startup complete.
logging1-1 | INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
logging1-1 | [logging1] stored 1775474197131948346 for user123
logging1-1 | INFO:      172.19.0.10:46736 - "POST /transactions HTTP/1.1" 200 OK
logging1-1 | [logging1] stored 1775474197434877888 for user456
logging1-1 | INFO:      172.19.0.10:46742 - "POST /transactions HTTP/1.1" 200 OK
logging1-1 | [logging1] stored 1775474197634417263 for user_b
logging1-1 | INFO:      172.19.0.10:46750 - "POST /transactions HTTP/1.1" 200 OK
logging1-1 | [logging1] read 1 tx for user001
logging1-1 | INFO:      172.19.0.10:46378 - "GET /transactions/user/user001 HTTP/1.1"
200 OK
```


Опис логів: - [logging1] connected to Hazelcast members=[...] - успішне підключення до Hazelcast кластера - [logging1] stored TransactionID for UserID - трансація збережена в розподіленій карті - [logging1] read N tx for user - читання трансацій користувача з Hazelcast - Інстанс logging1 обробляє запити та синхронізує дані між вузлами кластера

Результати тестування

Статус тестів:  6/6 пройдено

№	Тест	Результат	Деталі
1	Service Connectivity	 PASS	Всі сервіси доступні
2	Reset Services	 PASS	Успішне очищення даних
3	Transaction Flow	 PASS	Трансакція: user001 +100 = 100
4	Multiple Transactions	 PASS	3 трансакції: 50+100+150 = 300
5	Concurrent Users	 PASS	3 користувачі: 200+300+150 = 650
6	Metrics Collection	 PASS	12 викликів логування, 12 викликів балансів

Метрики продуктивності:

- **Logging avg:** 20.8 мс на запит
- **Counter avg:** 12.9 мс на запит
- **Total operations:** 12 успішних викликів кожного сервісу
- **Success rate:** 100%

Інструкції по запуску

Переддумови:

- Docker & Docker Compose
- Python 3.11+
- Python пакет: `requests`

1. Запуск сервісів

```
cd 03-microservices-with-hazelcast

# Очищення та перебудова
docker compose down --remove-orphans
docker compose up -d --build

# Перевірка статусу
docker compose ps
```

2. Запуск тестів

```
# Установка залежностей
pip install --break-system-packages requests

# Запуск full test suite
python3 e2e_test.py
```

3. Мануальне тестування API

```
# Приклад 1: Створення транзакції
curl -X POST http://localhost:8000/transaction \
  -H "Content-Type: application/json" \
  -d '{"user_id":"user001","amount":500,"message":"Deposit"}'

# Приклад 2: Отримання користувача
curl http://localhost:8000/user/user001

# Приклад 3: Отримання метрик
curl http://localhost:8000/metrics

# Приклад 4: Отримання балансів
curl http://localhost:8002/balances
```

4. Перегляд логів

```
# Всі логи
docker compose logs -f

# Логи конкретного сервісу
docker compose logs -f facade-service
docker compose logs -f counter-service
docker compose logs -f logging1

# Останні 50 рядків
docker compose logs --tail=50 facade-service
```

5. Ремонт системи

```
# Скидання всіх даних
docker compose down --volumes

# Перезапуск конкретного сервісу
docker compose restart counter-service

# Перебудова зображення
docker compose build --no-cache
docker compose up -d
```

Властивості системи

✓ Забезпечені властивості:

- **Масштабованість:** Кількість мікросервісів може збільшуватись незалежно
- **Надійність:** Трьохрівневий Hazelcast кластер для високої доступності
- **Постійність:** PostgreSQL для постійного збереження балансів
- **Моніторинг:** Вбудовані метрики для відстеження продуктивності
- **Розподіленість:** Логування транзакцій в Hazelcast для синхронізації між сервісами
- **HTTP-only:** Всі сервіси спілкуються через HTTP (без gRPC)



Забезпечена безпека:

- Docker контейнери для ізоляції
- Внутрішня мережа Docker для комунікації
- PostgreSQL з автентифікацією (user: lab3, pass: lab3)
- Keine sensitive дані в логах

Висновки

Система успішно демонструє:

1. ✓ **Мікросервісну архітектуру** - розділення відповідальності між сервісами
2. ✓ **Розподілене кешування** - Hazelcast для синхронізації даних
3. ✓ **Масштабованість** - горизонтальне розширення через додаткові інстанси
4. ✓ **Надійність** - резервування трьохма Hazelcast вузлами
5. ✓ **Моніторинг** - вбудовані метрики та логи для діагностики